Correzione Laboratorio 6 - 23.05.2022

Problema 1

Gli script Matlab associati allo svolgimento degli Esercizi 1 e 2 del Problema 1 si trovano nella cartella `CG_FEM_Lab6_P1_Soluzione.zip`.

Esercizio 1

1. L'implementazione della risoluzione del problema di Poisson con condizioni di Dirichlet vede la modifica delle seguenti `function`: `C_main2D.m` e `C_bound_cond2D.m` (che si trova nella cartella `BoundaryConditions`). Di seguito si riportano i testi delle due funzioni. In particolare sono stati scritti i comandi alle righe `71,77` del file `C_main2D.m` e alla riga `43` del file `C_bound_cond2D.m`.

2. Per rappresentare la soluzione numerica del problema sfruttiamo la `function` fornita nella cartella `Postprocessing`: `C_postprocessing.m`. La chiamata di questa funzione avviene nella `function C_main2D.m` alla riga `83`.

3. Per calcolare gli errori $L^2(\Omega)$ e $H^1(\Omega)$ associati alla soluzione numerica del problema sfruttiamo la `function C_compute_errors.m` nella cartella `Errors`. La chiamata di questa funzione avviene nella `function C_main2D.m` alla riga `89`.

```
1  function [errors,solutions,femregion,Matrices,Dati]=C_main2D(TestName,nRef)
2  %% [errors,solutions,femregion,Matrices,Dati]=C_main2D(TestName,nRef)
3  %==========================================================================
4  % Solution of the Poisson's problem with linear finite elements
5  % (non homogeneous Dirichlet boundary conditions)
6  %==========================================================================
7  %
8  %    INPUT:
9  %          Dati        : (struct)  see C_dati.m
10 %          nRef        : (int)     refinement level
11 %
12 %    OUTPUT:
13 %          errors      : (struct) contains the computed errors
14 %          solutions   : (sparse) nodal values of the computed and exact
15 %                        solution
16 %          femregion   : (struct) infos about finite elements
```

```
%                           discretization
%           Matrices     : (struct) fe stiffness and mass matrices
%           Dati         : (struc)  see C_dati.m
%

warning off;
addpath MeshGeneration
addpath FESpace
addpath BoundaryConditions
addpath Assembly
addpath Postprocessing
addpath Errors

%==========================================================================
% LOAD DATA FOR TEST CASE
%==========================================================================

Dati = C_dati(TestName);
Dati.nRefinement = nRef;

%==========================================================================
% MESH GENERATION
%==========================================================================

[Region] = C_create_mesh(Dati);

%==========================================================================
% FINITE ELEMENT REGION
%==========================================================================

[femregion] = C_create_femregion(Dati,Region);

%==========================================================================
% BUILD FINITE ELEMENT MATRICES and RIGHT-HAND SIDE
%==========================================================================

[A,b] = C_matrix2D(Dati,femregion);

%==========================================================================
% IMPOSE BOUNDARY CONDITIONS
%==========================================================================

[A,b,u_g] = C_bound_cond2D(A,b,femregion,Dati);

%==========================================================================
% SAVE DATA in THE STRUCT Matrices
%==========================================================================

[Matrices] = C_build_out_matrices(A,b,u_g);

%==========================================================================
% SOLVE THE LINEAR SYSTEM
```

```
%==========================================================================

u0 = A\b;

%==========================================================================
% ASSIGN DIRICHLET BOUNDARY CONDITIONS
%==========================================================================

uh = u0 + u_g;

%==========================================================================
% POST-PROCESSING OF THE SOLUTION
%==========================================================================

[solutions] = C_postprocessing(Dati, femregion, uh);

%==========================================================================
% ERROR ANALYSIS
%==========================================================================

[errors] = C_compute_errors(Dati, femregion, solutions);
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Imposizione delle condizioni al contorno per il problema di
% Poisson non omogeneo.
% INPUT: A matrice di stiffness
%        b vettore termine noto
%        femregion struttura contenente la matrice metrica
% OUTPUT: A,b modificate secondo le condizioni al bordo
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

function [A,b,u_g] = C_bound_cond2D(A,b,femregion,Dati)
%% [A,b,u_g] = C_bound_cond2D(A,b,femregion,Dati)
%==========================================================================
% Assign Dirchlet boundary conditions
%==========================================================================
%    called in C_main2D.m
%
%    INPUT:
%          A            : (sparse(ndof,ndof) real) stiffness matrix
%          b            : (sparse(ndof,1) real) rhs vector
%          femregion    : (struct)  see C_create_femregion.m
%          Dati         : (struct)  see C_dati.m

%
%    OUTPUT:
%          A            : (sparse(ndof,ndof) real) stiffness matrix
%          b            : (sparse(ndof,1) real) rhs vector
%          u_g          : (sparse(ndof,1) real) evaluation of Dirichlet
    conditions
%

```

```
fprintf('==================================================================\n')
fprintf('Assign Dirichlet boundary conditions ... \n');
fprintf('==================================================================\n')

ndof = length(b);
u_g = sparse(ndof,1);

boundary_points = femregion.boundary_points;
x = femregion.dof(boundary_points,1);
y = femregion.dof(boundary_points,2);
u_g(boundary_points) = eval(Dati.exact_sol);

A_0 = A;
b_0 = b - A*u_g; % rilevamento

for k = 1:length(boundary_points)
    A_0(boundary_points(k),:) = 0;
    A_0(:,boundary_points(k)) = 0;
    A_0(boundary_points(k),boundary_points(k)) = 1;
    b_0(boundary_points(k)) = 0;
end

b = b_0;
A = A_0;
```

Esercizio 2

Per i dettagli si vedano i file `Lab6_Es2.m` e `C_dati.m`. Si riportano di seguito gli errori calcolati ed i corrispondenti ordini di convergenza p.

h	0.1768	0.0884	0.0442	0.0221
$\|u - u_h\|_{L^2(\Omega)}$	0.0850	0.0227	0.0058	0.0014
$\|u - u_h\|_{H^1(\Omega)}$	1.6772	0.8637	0.4351	0.2180

Tabella 1: Errori $H^1(\Omega)$ e $L^2(\Omega)$ calcolati come funzione della dimensione di griglia h.

$p_{L^2(\Omega)}$	1.9083	1.9766	1.9941
$p_{H^1(\Omega)}$	0.9575	0.9892	0.9973

Tabella 2: Ordini di convergenza $H^1(\Omega)$ e $L^2(\Omega)$.

Problema 2

Gli script Matlab associati allo svolgimento degli Esercizi 3 e 4 del Problema 2 si trovano nella cartella `CG_FEM_Lab6_P2_Soluzione.zip`.

Esercizio 1

1. L'implementazione della matrice di massa globale (sfruttando la `function C_mass_loc.m`) vede la modifica della `function C_matrix2D.m` (che si trova nella cartella `Assembly`). Di seguito si riporta il testo della funzione. In particolare sono stati scritti i comandi alle righe `63-64` del file `C_matrix2D.m`.

2. Le modifiche per risolvere il Problema 2 nel dominio Ω con condizioni di Dirichlet g, funzione f e parametri α, β (costanti) assegnati sono state effettuate nelle `function C_main2D.m`, `C_matrix2D.m` e `C_bound_cond2D.m` (nella cartella `BoundaryConditions`). Nella `function C_matrix2D.m` é stata implementata la matrice del problema P alle righe `73,74`. Nella `function C_bound_cond2D.m` sono state inserite in modo opportuno le condizioni di Dirichlet alla riga `49`. Nella `function C_main2D.m` é stata utilizzata la matrice del problema P per risolvere il sistema lineare alla riga `71`.

```
1  function [A,f,M,P]=C_matrix2D(Dati,femregion)
2  %% [A,f,M] = C_matrix2D(Dati,femregion)
3  %==========================================================================
4  % Assembly of the stiffness matrix A, rhs f and mass matrix M
5  %==========================================================================
6  %    called in C_main2D.m
7  %
8  %    INPUT:
9  %          Dati        : (struct)  see C_dati.m
10 %          femregion   : (struct)  see C_create_femregion.m
11 %
12 %    OUTPUT:
13 %          A           : (sparse(ndof,ndof) real) stiffnes matrix
14 %          f           : (sparse(ndof,1) real) rhs vector
15 %          M           : (sparse(ndof,ndof) real) mass matrix
16 %          P           : (sparse(ndof,ndof) real) problem matrix
17 %
18
19 warning off;
20 addpath FESpace
21 addpath Assembly
22
23 fprintf('==========================================================\n')
24 fprintf('Assembling matrices and right hand side ... \n');
25 fprintf('==========================================================\n')
26
27
28 % connectivity infos
29 ndof        = femregion.ndof;
30 nln         = femregion.nln;
31 ne          = femregion.ne;
32 connectivity = femregion.connectivity;
33
34
35 % shape functions
```

```
[basis] = C_shape_basis(Dati);

% quadrature nodes and weights for integrals
[nodes_2D, w_2D] = C_quadrature(Dati);

% evaluation of shape bases
[dphiq,Grad] = C_evalshape(basis,nodes_2D);


% Assembly begin ...
A = sparse(ndof,ndof);
M = sparse(ndof,ndof);
f = sparse(ndof,1);

for ie = 1 : ne

    iglo = connectivity(1:nln,ie);

    % Determinant of the Jacobian of the map and physical coordinates
    [BJ, pphys_2D] = C_get_Jacobian(femregion.coord(iglo,:), nodes_2D, Dati.
   MeshType);

    %=================================================================%
    % STIFFNESS MATRIX AND MASS MATRIX
    %=================================================================%
    [A_loc] = C_lap_loc(Grad,w_2D,nln,BJ);
    A(iglo,iglo) = A(iglo,iglo) + A_loc;

    [M_loc] = C_mass_loc(dphiq,w_2D,nln,BJ);
    M(iglo,iglo) = M(iglo,iglo) + M_loc;

    %=================================================
    % FORCING TERM -- RHS
    %=================================================
    [load] = C_loc_rhs2D(Dati.force,dphiq,BJ,w_2D,pphys_2D,nln);
    f(iglo) = f(iglo) + load;
end

P = sparse(ndof,ndof);
P = Dati.alpha*A + Dati.beta*M;
```

```
function [errors,solutions,femregion,Matrices,Dati]=C_main2D(TestName,nRef)
%% [errors,solutions,femregion,Matrices,Dati]=C_main2D(TestName,nRef)
%=================================================================
% Solution of the Poisson's problem with linear finite elements
% (non homogeneous Dirichlet boundary conditions)
%=================================================================
%
%    INPUT:
%          Dati        : (struct)  see C_dati.m
%          nRef        : (int)     refinement level
%
```

```
%     OUTPUT:
%           errors      : (struct) contains the computed errors
%           solutions   : (sparse) nodal values of the computed and exact
%                         solution
%           femregion   : (struct) infos about finite elements
%                         discretization
%           Matrices    : (struct) fe stiffness and mass matrices
%           Dati        : (struc)  see C_dati.m
%

warning off;
addpath MeshGeneration
addpath FESpace
addpath BoundaryConditions
addpath Assembly
addpath Postprocessing
addpath Errors

%==========================================================================
% LOAD DATA FOR TEST CASE
%==========================================================================

Dati = C_dati(TestName);
Dati.nRefinement = nRef;

%==========================================================================
% MESH GENERATION
%==========================================================================

[Region] = C_create_mesh(Dati);

%==========================================================================
% FINITE ELEMENT REGION
%==========================================================================

[femregion] = C_create_femregion(Dati,Region);

%==========================================================================
% BUILD FINITE ELEMENT MATRICES and RIGHT-HAND SIDE
%==========================================================================

[A,b,M,P] = C_matrix2D(Dati,femregion);

%==========================================================================
% IMPOSE BOUNDARY CONDITIONS
%==========================================================================

[A,b,M,P,u_g] = C_bound_cond2D(A,b,M,P,femregion,Dati);

%==========================================================================
% SAVE DATA in THE STRUCT Matrices
%==========================================================================
```

```
[Matrices] = C_build_out_matrices(A,b,M,P,u_g);

%==========================================================================
% SOLVE THE LINEAR SYSTEM
%==========================================================================

u0 = P\b;

%==========================================================================
% ASSIGN DIRICHLET BOUNDARY CONDITIONS
%==========================================================================

uh = u0 + u_g;

%==========================================================================
% POST-PROCESSING OF THE SOLUTION
%==========================================================================

[solutions] = C_postprocessing(Dati,femregion,uh);

%==========================================================================
% ERROR ANALYSIS
%==========================================================================

[errors] = C_compute_errors(Dati,femregion,solutions);
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Imposizione delle condizioni al contorno per il problema di
% Poisson non omogeneo.
% INPUT: A matrice di stiffness
%        b vettore termine noto
%        M matrice di massa
%        P matrice del problema
%        femregion struttura contenente la matrice metrica
% OUTPUT: A,b,M,P modificate secondo le condizioni al bordo
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

function [A,b,M,P,u_g]=C_bound_cond2D(A,b,M,P,femregion,Dati)
%% [A,b,M,P,u_g]=C_bound_cond2D(A,b,M,P,femregion,Dati)
%==========================================================================
% Assign Dirchlet boundary conditions
%==========================================================================
%    called in C_main2D.m
%
%    INPUT:
%          A            : (sparse(ndof,ndof) real) stiffness matrix
%          b            : (sparse(ndof,1) real) rhs vector
%          femregion    : (struct)  see C_create_femregion.m
%          Dati         : (struct)  see C_dati.m

%
```

```
%    OUTPUT:
%          A            : (sparse(ndof,ndof) real) stiffness matrix
%          b            : (sparse(ndof,1) real) rhs vector
%          M            : (sparse(ndof,ndof) real) mass matrix
%          P            : (sparse(ndof,ndof) real) problem matrix
%          u_g          : (sparse(ndof,1) real) evaluation of Dirichlet conditions
%

fprintf('==================================================================\n')
fprintf('Assign Dirichlet boundary conditions ... \n');
fprintf('==================================================================\n')

ndof = length(b);
u_g = sparse(ndof,1);

boundary_points = femregion.boundary_points;
x = femregion.dof(boundary_points,1);
y = femregion.dof(boundary_points,2);
u_g(boundary_points) = eval(Dati.exact_sol);

A_0 = A;
M_0 = M;
P_0 = P;
b_0 = b - P*u_g; %rilevamento


for k = 1:length(boundary_points)
    A_0(boundary_points(k),:) = 0;
    A_0(:,boundary_points(k)) = 0;
    A_0(boundary_points(k),boundary_points(k)) = 1;
    M_0(boundary_points(k),:) = 0;
    M_0(:,boundary_points(k)) = 0;
    M_0(boundary_points(k),boundary_points(k)) = 1;
    P_0(boundary_points(k),:) = 0;
    P_0(:,boundary_points(k)) = 0;
    P_0(boundary_points(k),boundary_points(k)) = 1;
    b_0(boundary_points(k)) = 0;
end

b = b_0;
A = A_0;
M = M_0;
P = P_0;
```

Esercizio 2

Per i dettagli si vedano i file `Lab6_Es3.m` e `C_dati.m`.